RAGS: Reinforcement Learning-based Global Scheduling for LLM Agent Serving Systems

Anonymous Author(s)

Abstract

Modern LLM serving systems use a global scheduler to manage request queuing and route inference calls across backend instances. However, LLM-based Agent workloads form dynamic multi-step execution graphs, where inter-request dependencies, evolving execution progress, and cache locality must be considered jointly. These properties expose limitations in existing queuing and routing policies. In queuing, prioritizing workflows near completion can reduce end-to-end latency, but First-Come-First-Serve ignores execution progress, while static topology-aware strategies become unreliable when dynamic branches or loops invalidate runtime estimates. In routing, schedulers must balance KV cache reuse and global load distribution: round-robin sacrifices cache locality, whereas static session affinity causes hotspots during parallel fan-out and adapts poorly to load shifts.

To address these challenges, we present RAGS, an RL-based Agent Global Scheduler that jointly co-designs queuing and routing. Its queuing layer, DynA, requires no predefined topology and assigns priorities using an online-estimated completion probability derived from runtime step distributions, augmented by an aging mechanism to prevent starvation. Its routing layer formulates request assignment as a Markov Decision Process and uses a Transformer-based policy trained through deep reinforcement learning. Through offline pretraining and online fine-tuning, the policy jointly optimizes KV cache reuse, load balance, and queue congestion. Extensive evaluations on diverse Agent workloads in a simulated LLM serving cluster show that RAGS reduces mean end-to-end latency by up to 5.2x over round-robin and by 3–4x over state-of-the-art baselines. Additional studies further confirm the complementary effects of both layers and strong generalizability across heterogeneous GPU deployments.

CCS Concepts

• **Computing methodologies** → **Planning and scheduling**; • **Computer systems organization** → *Cloud computing*.

Keywords

LLM serving, Agent workflows, request scheduling, reinforcement learning

1 Introduction

Large Language Models (LLMs) have rapidly evolved from standalone research artifacts into the computational backbone of diverse intelligent applications. Modern LLM serving systems rely heavily on a global scheduler to manage request queuing and route inference calls across backend instances [10, 22]. However, these conventional schedulers are fundamentally designed for independent, single-turn requests, treating each inference call as an isolated scheduling unit.

LLM-based Agent systems inherently challenge this assumption. Applications such as multi-turn chatbots, autonomous code generation pipelines, and large-scale document processing workflows orchestrate multiple LLM calls into dynamic, multi-step execution graphs [21]. As these deployments scale to handle thousands of concurrent workflows, global scheduling must evolve beyond request-level efficiency. It must strictly account for workflow-level dependencies, dynamic execution progress, and the critical trade-off between KV cache locality and cluster-wide load balancing. Consequently, these structural properties expose severe limitations in existing global queuing and routing policies.

On the queuing side, Agent workflows diverge fundamentally from conventional single-turn workloads in both structure and objective. Because these workflows frequently incorporate conditional branches and iterative repair loops, their true execution paths and depths are resolved only at runtime. Consequently, static topology-aware priority estimation becomes highly unreliable whenever branch outcomes or loop iterations deviate from prior expectations. Traditional First-Come-First-Serve (FCFS) queuing is equally inadequate; by remaining entirely oblivious to workflow progress, it often stalls near-completion tasks and amplifies end-to-end latency along dataflow dependencies. Furthermore, because requests from different workflow stages exhibit substantial variance in context lengths, generation lengths, and GPU memory footprints, applying a uniform scheduling treatment inevitably induces head-of-line blocking and resource imbalance.

On the routing side, Agent workloads force schedulers to navigate a strict trade-off between two competing objectives: cache locality and load balancing. Because adjacent inference calls within the same workflow typically share overlapping context prefixes, they present significant opportunities for KV cache reuse [10, 23]. However, optimizing routing exclusively for cache locality frequently overloads specific backend instances, particularly during parallel fan-out stages. While conventional Round-Robin routing achieves uniform global load distribution, it completely sacrifices this prefix cache locality. Conversely, static session affinity, as adopted by systems like Parrot [11], maximizes cache reuse via fixed instance binding but inevitably triggers hardware hotspots and struggles to adapt to dynamic runtime load shifts.

In response to these challenges, we present **RAGS** (RL-based Agent Global Scheduler), a co-designed global scheduling framework that addresses both queuing and routing dimensions in a coordinated manner. The queuing layer, **DynA**, determines request priorities using exclusively runtime-observable signals (specifically, online-estimated completion-round distributions, accumulated workflow latencies, and per-request queue ages), operating independently of any predefined workflow topology. Concurrently,

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

the routing layer employs a Reinforcement Learning (RL) policy that models request-to-instance assignment as a Markov Decision Process. By training a Transformer-based network via Double-DQN, it jointly optimizes KV cache reuse, cluster load balance, and queue congestion, ensuring continuous adaptability through offline pre-training coupled with online fine-tuning.

The main contributions of this paper are as follows.

- We propose an adaptive queuing algorithm that estimates the overall completion probability of active workflows using statistics gathered at runtime. By prioritizing individual requests based on the execution progress of their parent workflows, this approach effectively mitigates head-of-line blocking among concurrent tasks.
- To the best of our knowledge, RAGS is the first to apply reinforcement learning to global routing in LLM Agent serving. By jointly encoding heterogeneous cluster features, including KV cache states, load distributions, and GPU hardware configurations, the RL policy realizes an adaptive, multi-objective routing strategy that strictly outperforms traditional static heuristics.
- Evaluations on a simulated LLM serving cluster demonstrate that RAGS reduces mean end-to-end workflow latency by up to **5.2**× compared to a normal Round-Robin baseline, and by **3–4**× against Parrot and Ayo. Furthermore, ablation studies and experiments on heterogeneous GPU clusters validate the individual effectiveness and hardware adaptability of the co-designed queuing and routing modules.

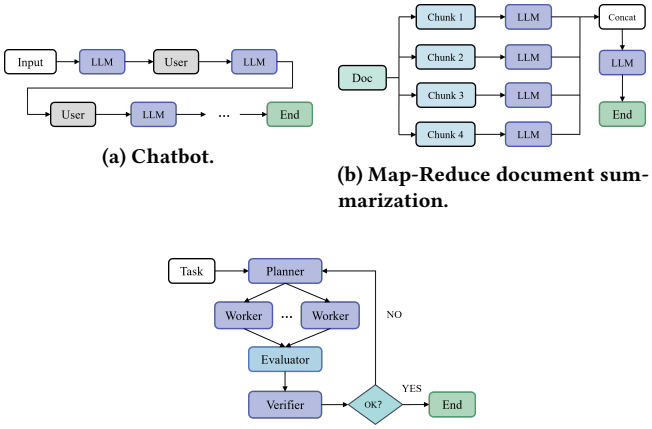
2 Background and Problem Analysis

Conventionally, LLM serving systems employ a layered scheduling architecture in which the global scheduler manages two interconnected operations: (1) determining the execution priority of pending requests, and (2) routing each request to an appropriate backend instance. This section characterizes the distinctive structural properties of LLM Agent workloads to demonstrate why conventional request-level heuristics prove inadequate for these tasks. By exposing these systemic limitations, we motivate the necessity of the comprehensive global scheduling framework detailed in Section 3, which systematically addresses both the queuing and routing dimensions.

2.1 Characteristics of LLM Agent Workflows

To systematically characterize LLM Agent workloads, we surveyed mainstream open-source projects on GitHub and analyzed representative paradigms, including chatbots, code generation, and document summarization. As illustrated in Figure 1, a typical Agent workflow can be abstracted as a directed graph. In this structural model, each node represents an individual LLM inference call or tool invocation, while each edge encodes a specific dataflow or control dependency. Based on this analysis, we identify three distinctive system-level traits inherent to Agent workloads.

Dynamic, nonlinear topologies with mixed concurrency. Agent workflows incorporate conditional branches, feedback loops, and interleaved sequential and parallel stages (e.g., iterative repair cycles and Map-phase fan-outs). Consequently, these features result



(a) Chatbot.

(b) Map-Reduce document summarization.

(c) Code generation.

Figure 1: Representative LLM Agent workflow structures.

in dynamically changing execution paths, rendering static graph-based scheduling approaches ineffective.

Heterogeneous inference costs across workflow nodes. Nodes serving different roles (such as routing, generation, and quality review) emit requests with varying prompt lengths, output lengths, and execution latencies. Because these resource requirements can differ by over 25× within a single application [5], scheduling decisions must be strictly cost-aware.

Deep context sharing and KV cache affinity. Adjacent calls within the same workflow contain overlapping context prefixes, presenting KV cache reuse opportunities. However, affinity-driven routing inherently concentrates requests and induces node-level hotspots, forcing a strict trade-off between cache locality and cluster-wide load balancing.

Taken together, these architectural traits inherently contradict the classical assumption of independent, homogeneous requests. The following two subsections examine the concrete limitations that arise when standard queuing and routing policies are applied to Agent workloads.

2.2 Limitations of Existing Queuing Algorithms

Conventional LLM serving paradigms model requests as independent, single-turn calls, where FCFS queuing is sufficient for fairness. Agent workloads, however, comprise interdependent requests that introduce three fundamental scheduling challenges.

(1) Head-of-line blocking from workflow progress heterogeneity. Concurrent Agent requests exhibit varying execution progress, meaning the system simultaneously processes initial LLM calls and near-terminal tasks. Because FCFS is progress-agnostic, it frequently stalls near-completion requests behind newly admitted early-stage tasks, directly inflating end-to-end workflow latencies.

(2) Cascading amplification and accumulation of queuing delays. Queuing delays experienced at any individual node propagate along dataflow dependencies. Furthermore, because FCFS ranks requests solely by arrival time, it ignores accumulated workflow-level latencies. Consequently, tail performance degrades as delays compound across successive stages.

(3) Inability to handle dynamic workflow topologies. Topology-aware strategies like Ayo [17] prioritize tasks with fewer remaining stages based on a static workflow graph. However, when dynamic branches or loops execute at runtime, the statically estimated residual stage counts deviate from actual execution paths, causing tasks within active loops to be improperly under-prioritized.

Collectively, these limitations necessitate an adaptive queuing policy that tracks actual workflow progress and accumulated latencies independent of static topologies.

2.3 Limitations of Existing Routing Strategies

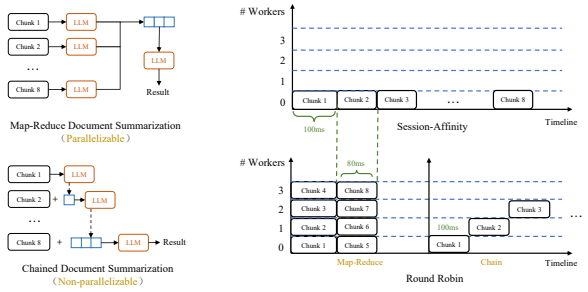


Figure 2: RR versus Session Affinity under parallel and sequential document-summarization Agent workflows.

Conventional global routing mechanisms for LLM serving typically utilize Round-Robin (RR) to distribute requests sequentially for load balancing, or Session Affinity (SA) to bind session requests to a fixed instance for maximizing KV cache reuse. However, both strategies exhibit structural limitations when processing Agent workloads.

RR ignores request heterogeneity. Because requests from distinct functional nodes exhibit diverse context lengths and GPU memory footprints, RR's uniform routing often concentrates memory-intensive tasks on a single instance. This concentration saturates GPU memory and triggers request preemptions. Consequently, preempted tasks incur re-queuing and recomputation overheads, directly degrading overall system throughput.

SA mishandles intra-workflow concurrency. During parallel execution stages (e.g., the Map phase in document summarization), workflows emit numerous concurrent subtask requests. By binding all these requests to a single instance, SA creates local hardware hotspots while leaving other instances underutilized, thereby compromising cluster-wide load balancing.

As illustrated in Figure 2, RR achieves lower latency on parallel workflows by exploiting cluster-wide concurrency, whereas SA outperforms RR on sequential workflows by preserving KV cache locality across stages. Because neither policy dominates universally, the optimal routing choice depends jointly on execution topologies, temporal dependencies, and real-time cache states.

To address these limitations, recent systems incorporate richer heuristics, such as multi-objective linear weighting. However, LLM cluster states evolve continuously: request arrival patterns, instance queue lengths, and memory utilization fluctuate rapidly. Furthermore, each routing decision reshapes the environment encountered by subsequent requests. Static, fixed-weight heuristics lack the agility to capture these complex state transitions or to optimize the trade-off between immediate scheduling gains and long-horizon throughput.

Collectively, the isolated failures of existing queuing and routing algorithms necessitate the holistic global scheduling framework detailed in Section 3.

3 Algorithm Design

3.1 System Architecture

Figure 3 illustrates the overall system architecture and request processing pipeline of RAGS. The system frontend accepts heterogeneous Agent workflows and forwards all LLM inference requests to the global scheduler for centralized management. Within the scheduler, requests first enter the global queue, where the adaptive queuing module, **DynA**, calculates priorities and reorders pending tasks. Subsequently, the RL-driven routing policy asynchronously assigns these requests to target instances by evaluating queue characteristics alongside cluster states. Following inference completion, the system triggers subsequent workflow stages. Concurrently, backend instances continuously report load metrics back to the scheduler to maintain an accurate environmental state. Once the workflow concludes, the system returns the final result to the user.

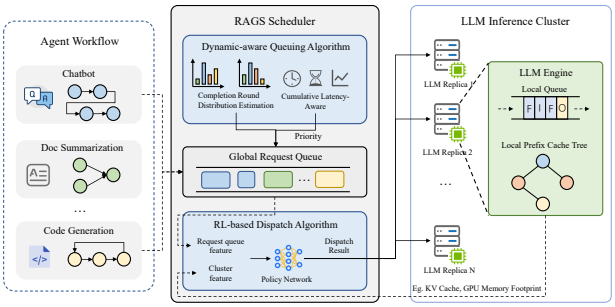


Figure 3: RAGS system architecture.

Building upon this architectural overview, the following subsections detail the two core scheduling components: the queuing algorithm and the routing policy.

3.2 DynA: Dynamic-Aware Queuing Algorithm

Due to the dynamic branching and autoregressive uncertainty inherent in Agent workflows, traditional queuing policies that rely on static topologies prove inadequate. These methods assume a priori knowledge of the workflow structure, a condition rarely satisfied in actual deployments.

To overcome this reliance, we introduce **DynA** (Dynamic-Aware Queuing). The core principle of DynA is to derive all priority decisions exclusively from runtime-observable signals, including online-estimated completion-round distributions, accumulated workflow latencies, and per-request queue ages. By doing so, it operates entirely independently of predefined workflow topologies. Specifically,

the algorithm functions across two dimensions: inter-Agent priority ordering governed by actual workflow progress, and intra-Agent request ordering driven by accumulated latencies.

3.2.1 Progress-Driven Inter-Agent Prioritization. To distinguish the execution progress of varying Agent types and prioritize requests nearing completion, DynA introduces an online completion-round distribution mechanism. This approach evaluates macroscopic workflow progress without requiring any a priori knowledge of the underlying topology.

Let $F_a(n)$ denote the empirical CDF of completion rounds for Agent type a. Specifically, this defines the probability that the total number of LLM inference calls within a completed workflow instance of this type does not exceed n. The system maintains this distribution dynamically by aggregating statistics from finished workflows at runtime, effectively bypassing the need for static structural assumptions.

Specifically, upon the completion of any Agent workflow of type a, the system records its completion round count n_j and updates the empirical CDF $F_a(n)$ using an Exponential Moving Average (EMA):

$$F_a^{(t+1)}(n) = (1-\lambda) \cdot F_a^{(t)}(n) + \lambda \cdot \mathbb{1}[n_j \le n], \tag{1}$$

where $\lambda \in (0,1)$ denotes the smoothing coefficient and $\mathbb{1}[\cdot]$ is the indicator function. By weighting recent observations more heavily, this EMA mechanism allows $F_a(n)$ to continuously adapt to workload shifts while gracefully decaying the influence of historical data.

For Agent types lacking historical records, the system initializes F_a with a uniform prior over the interval $[1, N_{\max}]$:

$$F_a(n) = \begin{cases} 0, & n < 1 \\ n/N_{\max}, & 1 \le n \le N_{\max} \\ 1, & n > N_{\max} \end{cases}.$$

During online execution, the EMA mechanism continuously drives this prior toward the actual workload distribution.

Building upon this dynamically updated CDF, when the scheduler evaluates a request from Agent type a that has already completed k LLM calls, it calculates the conditional completion probability $\hat{P}_{a,k}$. This metric quantifies the likelihood that the workflow will terminate within the next Δ steps, given that k steps have already been executed:

$$\hat{P}_{a,k} = P(k \le N_a \le k+\Delta \mid N_a \ge k) = \frac{F_a(k+\Delta) - F_a(k-1)}{1 - F_a(k-1)}, \tag{2}$$

where N_a denotes the random variable representing total LLM calls for Agent type a, and Δ defines a look-ahead window (default Δ=1). Because a larger $\hat{P}_{a,k}$ signifies an imminent completion, DynA elevates the scheduling priority of such requests. This design effectively approximates the Shortest Job First (SJF) policy, operating independent of predefined workflow topologies.

3.2.2 Intra-Agent Ordering via Cumulative Latency. To resolve scheduling ties among concurrent requests of the same Agent type and completion round, DynA applies an intra-Agent ordering strategy driven by workflow start times. Let t_r^{start} denote the initial invocation time of the workflow containing request r. The system prioritizes requests with earlier t_r^{start} values, compensating requests that have accumulated greater end-to-end delays.

To prevent the starvation of long-horizon workflows under this priority scheme, DynA incorporates an aging mechanism based on per-request wait times. Let $\tau_r = t_{\text{current}} - t_r^{\text{arrive}}$ define the current queuing delay for request r. Once τ_r exceeds a predefined maximum tolerance threshold $\tau_{\max}$, the system triggers a proportional priority boost:

$$\text{Boost}(r) = \max\left(0, \ \frac{\tau_r - \tau_{\max}}{\tau_{\max}}\right). \tag{3}$$

Integrating the conditional completion probability with this aging mechanism, DynA formulates a unified priority score for each queued request r:

$$\text{Score}(r) = \hat{P}_{a_r,k_r} + \eta \cdot \text{Boost}(r), \tag{4}$$

where a_r identifies the Agent type of r, k_r tracks the executed LLM calls for its workflow instance, and η serves as the aging weight hyperparameter. Requests with higher scores are scheduled first, with any remaining ties broken by t_r^{start}. Algorithm 1 summarizes the complete DynA policy.

Algorithm 1 Dynamic-Aware Queuing Algorithm (DynA)

Require: Request queue Q; online completion-round distributions $\{F_a\}$; anti-starvation threshold $\tau_{\max}$; weight η
Ensure: Next request to route r^*

```
for each request r ∈ Q do
    Retrieve Agent type a_r and completed call count k_r
    if F_{a_r} is not yet initialized then
        F_{a_r}(n) ← min(n, N_max)/N_max    // uniform prior
    end if
    P̂_{a_r,k_r} ← (F_{a_r}(k_r+Δ) − F_{a_r}(k_r−1)) / (1 − F_{a_r}(k_r−1))
    τ_r ← t_current − t_r^arrive
    Boost(r) ← max(0, (τ_r − τ_max)/τ_max)
    Score(r) ← P̂_{a_r,k_r} + η · Boost(r)
end for
Sort all requests in Q by (Score(r) desc, t_r^start asc)
r* ← first request in Q
return r*
```

3.3 RL-based Routing Algorithm

3.3.1 Problem Formulation and MDP Definition. We formalize the global scheduler's routing process as a Markov Decision Process (MDP) and employ reinforcement learning to derive an optimal routing policy.

Consider a service system comprising K LLM instances, each deploying model M on associated GPU hardware G. During continuous system operation, N Agent workflows arrive dynamically. At any decision step t, let Q_t denote the set of pending requests within the global scheduler. For each request $q_{i:k}$ (representing the k-th inference call of workflow w_i), the scheduler must assign a target instance $a_t \in \{1, \ldots, K\}$. The fundamental optimization objective is

to minimize the average end-to-end latency across all N workflows:

$$\min_{a_t} \frac{1}{N} \sum_{i=1}^{N} l_i, \tag{5}$$

where l_i defines the actual end-to-end completion time of workflow w_i.

Since Q_t often contains multiple concurrent requests, a centralized joint routing decision would necessitate an action space dimension of $O(|Q_t| \times K)$. Such a high-dimensional output space significantly impedes RL training efficiency and convergence. To resolve this, we adopt a sequential decision-making paradigm: the agent evaluates and routes pending requests individually.

Formally, the MDP operates as follows. At every interval Δt, the global scheduler sequentially processes all requests residing in the waiting queue. For the j-th request r_j during time step t, the agent observes the localized state $s_{t:j}$ and executes a discrete action $a_t^j \in \{1, \ldots, K\}$, effectively routing r_j to the corresponding LLM instance.

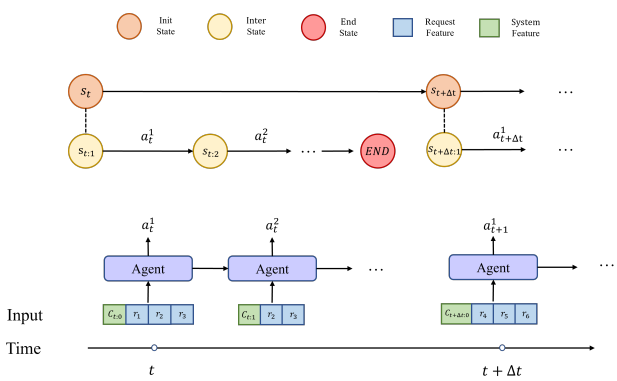


Figure 4: Markov Decision Process for global routing.

3.3.2 State Space. The state s_t encodes the current global cluster state, comprising features from the pending request queue and the real-time load of each LLM instance

Request state. The request feature matrix $\mathbf{X}_t^{req}$ consists of four distinct vectors:

- $\vec{p}$ is the input length vector, where p_i denotes the input token count for the i-th queued request.
- $\vec{d}$ is the estimated output length vector, where d_i represents the predicted generation length of the i-th request.
- $\vec{u}$ is the concurrency indicator vector, where $u_i \in \{0, 1\}$ flags whether the i-th request belongs to a parallel workflow stage.
- $\vec{\kappa}$ is the workflow type vector, where κ_i identifies the Agent application class of the i-th request.

To determine the estimated output length d_i, we adopt the methodology from [9], predicting discrete length intervals rather than exact token counts. This coarse-grained approximation is highly effective because the routing MDP relies strictly on relative interval magnitudes. Empirical evaluations confirm that this interval prediction achieves approximately 70% accuracy.

Cluster state. The instance feature matrix $\mathbf{X}_t^{inst}$ is constructed by concatenating individual vectors of all LLM instances. Specifically, the runtime state of each backend instance is characterized by the following components:

- $\vec{m}$ and $\vec{g}$: One-hot encoded vectors identifying the deployed model architecture and the specific GPU hardware configuration, respectively.
- $\vec{wq}$: A load histogram of the waiting queue, generated by binning input lengths using a configurable step size.
- $\vec{rq}$: A load histogram of active execution queues. Here, requests in the prefill phase are binned by input length, while those in the decode phase are binned by estimated output length.
- c: The GPU memory utilization, defined as the ratio of allocated KV cache blocks to total block capacity.
- s: The expected prefix cache reuse rate for the pending request, quantified by the fraction of reusable tokens.
- b: The estimated batch execution latency, tracked via a sliding window average of recent iteration times.

To ensure RL training stability, all extracted features are strictly normalized. The comprehensive MDP state representation is thus formulated as $s_t = [\mathbf{X}_t^{req} \parallel \mathbf{X}_t^{inst}]$, where $\parallel$ denotes the concatenation operator.

3.3.3 Reward Function. Using end-to-end workflow latency directly as a reward signal introduces severe reward sparsity and credit assignment challenges, given the asynchronous nature of LLM inference. To circumvent this, we design a composite immediate reward that provides dense feedback after every routing decision:

$$r_t = \lambda_k \cdot r_k + \lambda_{imb} \cdot r_{imb} + \lambda_p \cdot r_p + \lambda_T \cdot T(x_{t,i}^{inst}), \tag{6}$$

where the individual components are defined as follows:

- r_k: *Prefix cache reuse reward*, representing the expected KV cache hit rate to encourage a lower Time-to-First-Token (TTFT).
- r_{imb}: *Load-imbalance penalty*, quantifying the routing decision's impact on global load uniformity.
- r_p: *Queue penalty*, reflecting the target instance's waiting queue length to discourage routing to overloaded nodes.
- $T(x_{t,i}^{inst})$: *Throughput estimate*, denoting the expected token throughput of the selected instance given its current state.

To compute the load imbalance penalty ($r_{imb} = -S_{imb}$), we first estimate the total load L_i of instance i by consolidating its active ($\vec{rq}$) and waiting ($\vec{wq}$) queue distributions:

$$L_i = \sum_{j=1}^{N_{bin}} C_j \cdot (N_{run,j} + N_{pend,j}), \tag{7}$$

where C_j is the center length of bin j, while $N_{\cdot,j}$ is the count of requests within that bin. The normalized imbalance score S_{imb} is then calculated as:

$$S_{imb} = \frac{\sum_{i=1}^{K} |L_i - \bar{L}|}{\sum_{i=1}^{K} L_i + \epsilon}, \tag{8}$$

where $\bar{L} = \frac{1}{K} \sum_i L_i$ acts as the average cluster load, and ϵ ensures numerical stability.

For the throughput term $T(x_{t,i}^{inst})$, we employ a lightweight neural surrogate model trained offline on historical routing trajectories, which achieves a relative prediction error of approximately 10%.

3.3.4 Policy Network Architecture. To capture complex temporal dependencies and heterogeneous instance features, we introduce a Transformer-based [19] scheduling network that jointly encodes the target request, the waiting queue context, and the runtime state of each instance. Figure 5 illustrates the network topology.

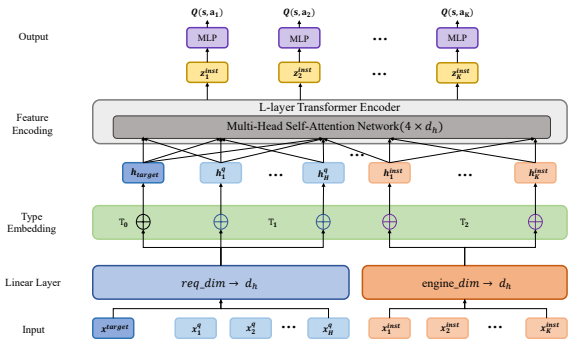


Figure 5: Routing policy network architecture.

The input layer comprises three feature subspaces: the target request $\mathbf{x}^{target}$, a look-ahead waiting queue $\{\mathbf{x}_i^q\}_{i=1}^H$ of length H, and the dynamic states of K LLM instances $\{\mathbf{x}_j^{inst}\}_{j=1}^K$. Each feature vector is first mapped to a common hidden dimension d_h via a linear projection, and subsequently augmented with learned type embeddings to distinguish target, queue, and instance tokens (type $\in \{0, 1, 2\}$). The resulting sequence is processed by L Transformer encoder layers to capture cross-token interactions. At the output layer, we extract the K instance-token representations $\{\mathbf{z}_{inst,j}\}_{j=1}^K$ and map each to an action-value score via a shared parameter Q-value head:

$$Q(s, a_j) = \mathbf{W}_Q \cdot \mathrm{ReLU}\big(\mathbf{W}_H \mathbf{z}_{inst,j} + \mathbf{b}_H\big) + b_Q. \tag{9}$$

The final output $\mathbf{Q} \in \mathbb{R}^K$ denotes the expected cumulative return for routing the current request to each instance.

3.3.5 Reinforcement Learning Training. We adopt a two-phase training paradigm that combines offline pretraining with online fine-tuning.

Offline Pretraining. We construct a data-driven offline simulation environment by replaying diverse real production workload traces at various load intensities, ensuring robust policy generalization.

To effectively navigate the discrete action space, we employ the Double-DQN algorithm [18], which decouples target action selection from value estimation to prevent Q-value overestimation. Furthermore, we replace the standard MSE loss with Huber Loss, which applies linear penalty to large TD errors, significantly improving training stability under heterogeneous request sequences.

Online Fine-tuning. Because static policies inevitably degrade in highly dynamic production environments, we deploy an online fine-tuning mechanism to iteratively optimize the agent. The global scheduler maintains a continuous feedback loop by logging runtime transitions into an experience replay buffer. Once sufficient samples are accumulated, the system initiates parameter updates at a low learning rate. To guarantee Quality of Service (QoS) stability during this live adaptation, we strictly bound the ϵ-greedy exploration rate and employ momentum-based soft updates for the target network, smoothing potential Q-value oscillations caused by workload distribution shifts.

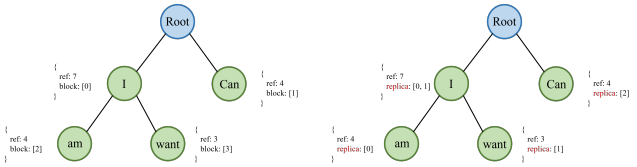


Figure 6: KV cache prefix trees: local (left) and global (right).

4 Implementation

To support RL agent training and system evaluation, we build our testbed upon Vidur [1], an open-source LLM inference simulator. Vidur natively supports heterogeneous GPUs (e.g., A100, H100) and provides built-in benchmarks for mainstream models like Llama and Qwen, enabling rapid validation of scheduling algorithms across diverse configurations.

However, native Vidur primarily targets single-turn LLM requests. It lacks the capability to simulate the multi-step interaction dependencies inherent in Agent workflows, nor does it support prefix-cache reuse. To address these limitations, we augment Vidur's core engine along two dimensions:

4.1 KV Cache Reuse

To enable KV cache reuse within the LLM engine (i.e., the local scheduler), we implement a `tree_cache` module atop Vidur's native `memory_pool`. This module comprises two core components: a *Block Manager* responsible for the allocation and reclamation of memory blocks, and a *Local Prefix Cache Tree* that maintains the prefix-mapping relationships of KV caches within each LLM instance.

Unlike sglang's single-level prefix tree, our design maintains both a per-instance local tree and a cluster-wide global tree augmented with replica mappings to enable cross-instance KV cache reuse. As shown in Figure 6, different request prefixes branch from the root node. Each node stores the token sequence, a reference count (`ref`) for memory reclamation, and a block index (`block`) mapping to the KV store.

To integrate with PagedAttention, only block-aligned prefixes are treated as valid cache entries, ensuring consistency with the underlying paged memory manager.

The *global KV cache prefix tree*, maintained by the global scheduler, records cache states across all LLM instances. Its structure mirrors the local tree, with the sole difference that each node stores a mapping from tree nodes to LLM instances (`replica`). When a user request arrives, the global scheduler can quickly locate matching instances through this tree, enabling cross-instance KV cache reuse.

4.2 Agent Workflow Support

Since Vidur cannot directly simulate complex Agent workflow logic, we extend it with a `UnifiedRequest` class that inherits from `Request` and adds `agent_type` and `agent_id` metadata for workflow identification.

`UnifiedRequest` further encodes a *workflow graph* of call-chain dependencies: subsequent stage requests are activated only after all predecessor requests in the graph have completed, faithfully simulating serial and parallel multi-step Agent interactions.

5 Evaluation

5.1 Experimental Setup

5.1.1 Hardware and Software Environment. All model training and experimental evaluation are conducted on a physical server running Ubuntu 18.04.6 LTS, equipped with dual Intel Xeon E5-2680 v3 processors (2.50 GHz) and two NVIDIA GeForce RTX 3090 GPUs.

For inference simulation we use the Vidur simulator. We emulate a four node LLM inference cluster in which each node is equipped with a 40 GB NVIDIA A100 GPU and independently deploys one Meta-Llama-3-8B instance by default. The prefill chunk size (`chunk_size`) is set to 4096 and the maximum batch capacity (`batch_size`) to 512.

Agent workflows are implemented with the LangChain framework [4] to generate realistic request sequences and workflow topologies for the experiments.

5.1.2 RL Training Configuration. We set the learning rates for offline pretraining and online fine-tuning to 5×10^{-5} and 1×10^{-5}, respectively, with a discount factor $\gamma = 0.995$. The experience replay buffer is configured with a maximum capacity of 50,000 samples and a warm-up period of 500 samples. During training, the policy network, which features a hidden dimension of 512, is optimized using mini-batches of 256 samples. To ensure stability, the target network is updated every 100 steps, while the ε-greedy exploration rate is linearly decayed from 0.5 to 0.01.

For DynA, we set the look-ahead window $\Delta = 1$, the anti-starvation threshold $\tau_{\max} = 10.0$ s, and $\eta = 0.1$.

5.1.3 Agent Workloads. To comprehensively evaluate the scheduling framework across diverse Agent topologies, we curate three representative LLM workloads featuring sequential, parallel, and hybrid execution graphs. In our evaluation traces, workflows follow a Poisson arrival process to emulate realistic varying loads. Figure 7 shows the token-length and request-count distributions of each workload.

ChatBot (serial). Derived from the ShareGPT dataset [7], we build ~8,000 multi-turn dialogue workflows (2–20 rounds each). Context lengths follow a right-skewed distribution (mean 837.54 tokens, concentrated under 2,000), while generation lengths average 191.78 tokens, mirroring standard long-context/short-generation conversational patterns.

Map-Reduce Summarization (parallel). This Agent utilizes a Map phase to concurrently summarize document chunks and a Reduce phase for final aggregation. Using Wikidata [20], we construct ~1,000 workflows (2–50 requests each). The average context length is 833.51 tokens with a higher variance than the ChatBot, alongside an average generation length of 186.51 tokens.

Code Generation Agent (hybrid). We formulate a multi-stage pipeline (User → Planner → Workers → Evaluators → Verifier) with a feedback loop that triggers iterative repair upon evaluation failure. Using HumanEval [6] and MBPP [3], we extract ~1,000 workflows (2–50 requests each). Because iterative repairs accumulate extensive history, the mean context length spikes to 3541.76 tokens, and a mean generation length of 800.87 tokens.

5.2 Baselines

We compare RAGS against the following baselines.

Round-Robin. Serving as our naive baseline, this strategy pairs First-Come-First-Serve (FCFS) queuing with round-robin routing. It treats all inference requests independently, operating entirely without Agent workflow awareness.

Parrot [11]. A state-of-the-art LLM serving system that uses semantic variable abstraction to expose inter-request DAG dependencies to the serving layer. It employs session affinity routing to maximize KV-cache prefix reuse while aggregating similar requests to satisfy application-level performance goals. We faithfully replicate its FCFS queuing and session affinity routing algorithms in our evaluation.

Ayo [17]. A fine-grained LLM orchestration framework that prioritizes requests based on their topological depth in the workflow DAG, with round-robin routing. We isolate and implement only its topology-aware queuing and routing logic for comparison.

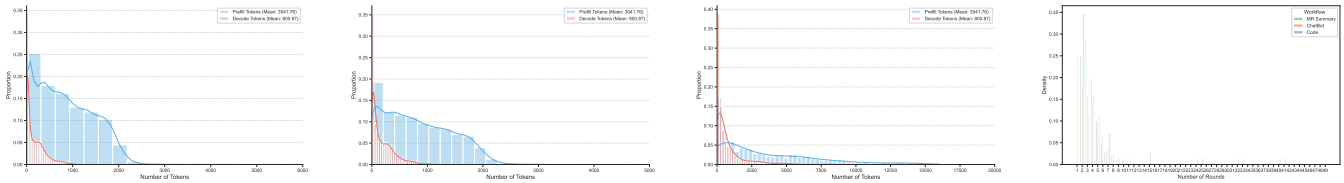
5.3 Main Results

Figure 8 presents the end-to-end (E2E) performance of each method under the three single-type workloads across varying arrival rates.

ChatBot (Sequential). At an arrival rate of 20 workflows/s, RAGS achieves up to a 5.2x reduction in mean E2E latency compared to Round-Robin and Ayo. Under high concurrency, Round-Robin's P50 TTFT degrades to 17 s, whereas RAGS maintains it below 1 s via RL-driven load balancing and DynA's priority queuing. Compared to Parrot, RAGS yields a marginal 1.1x speedup. Because ChatBot features a fixed topology with regular KV cache sharing, Parrot's static prefix routing already exploits cache locality effectively, leaving limited headroom for dynamic routing improvements.

Map-Reduce Summarization (Parallel). Because this workflow exhibits a simple two-level topology, performance gains are moderate. At 20 workflows/s, RAGS reduces mean E2E latency by roughly 6% over Round-Robin. Ayo's topology-based prioritization degenerates to basic round-robin because it cannot differentiate among same-level Map nodes. In contrast, DynA tracks actual workflow progress to prioritize requests nearing the Reduce aggregation, effectively alleviating synchronization bottlenecks.

Code Generation Agent (Hybrid). This workload incorporates autonomous repair loops to evaluate scheduling under dynamic topologies. Here, Parrot's static session affinity induces global load imbalances and queue congestion, resulting in a mean E2E latency of 240 s and a P95 tail of 600 s (roughly 3–4x higher than RAGS). By integrating RL-driven load balancing with DynA's progress-driven queuing, RAGS circumvents these overloaded bottlenecks. Consequently, it stabilizes mean E2E latency between 70–85 s and maintains a P95 latency around 200 s at 5 workflows/s.



(a) Token Distribution of ShareGPT Dataset **(b) Token Distribution of Wikidata Dataset** **(c) Token Distribution of CodeAgent Dataset** **(d) Request Count Distribution of Different Workflows**

Figure 7: Feature Distributions of Different Agent Workflows

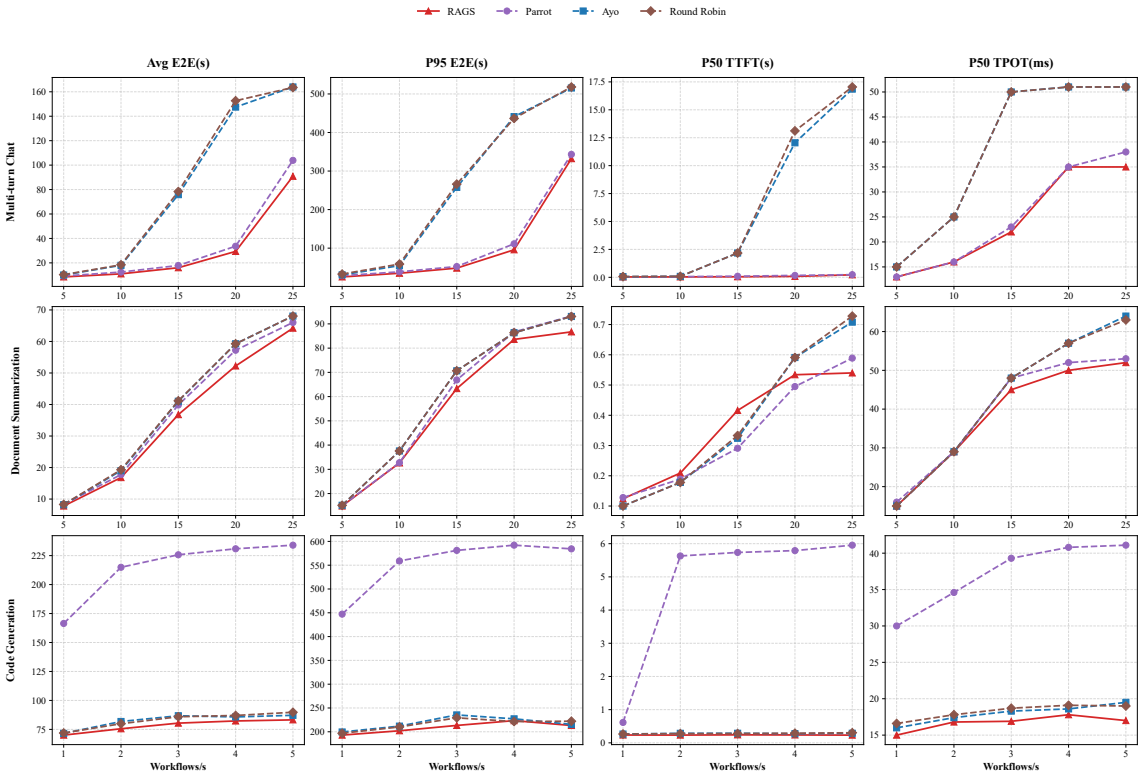


Figure 8: End-to-end performance comparison under three single-type workloads at varying request rates.

TTFT metrics further highlight queue congestion under dynamic loads. At 5 workflows/s, Parrot's P50 TTFT degrades to approximately 6 s. By jointly optimizing KV cache reuse and real-time load distribution, the holistic RAGS framework restricts P50 TTFT to under 0.5 s, demonstrating its resilience against dynamic workflow fluctuations.

Mixed workload. Production environments inherently process heterogeneous workflow mixtures. To evaluate system robustness under such conditions, we synthesize a dataset of 500 workflows, varying the composition ratio (x, y, z) of the ChatBot, Summarization, and Code Agent workloads at a constant arrival rate of 5 workflows/s. Figure 9 details the mean E2E latency across four representative mixture ratios.

Without Code Agent tasks (ratio $(0.5, 0.5, 0)$), baselines yield similar latencies (11–14 s), with RAGS achieving the optimal 10.6 s and Parrot 11.4 s. However, introducing Code Agent workflows (e.g., $(0, 0.5, 0.5)$ and $(0.5, 0, 0.5)$) degrades Parrot to ≈40 s, exposing static binding's inability to manage high compute density. Ayo and Round-Robin suffer similarly, proving static prioritization inadequate for heterogeneous traffic. Under the balanced mix $(0.33, 0.34, 0.33)$, RAGS achieves 18.7 s—outperforming Ayo by 14.5% and Parrot by 28.6%—and consistently bounds mean E2E latency below 22.8 s across all scenarios incorporating the Code Agent.

5.4 Hyperparameter Sensitivity

Figure 10 illustrates the impact of the load histogram's bin width on the mean E2E latency. Excessively narrow bin width results

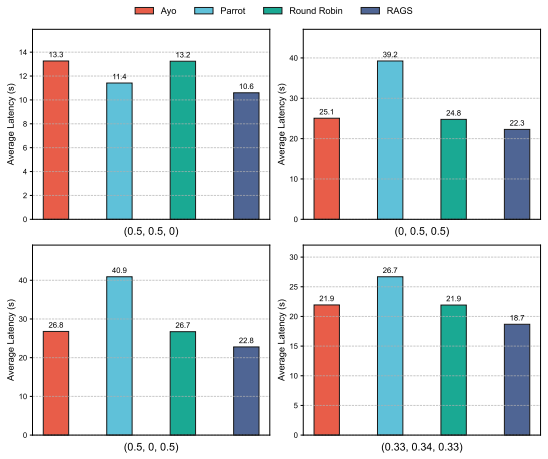


Figure 9: End-to-end performance comparison under mixed loads.

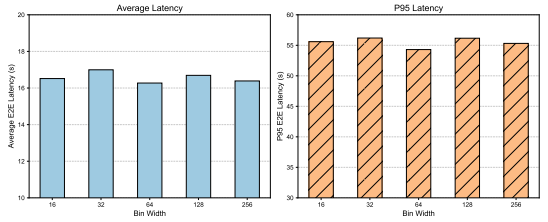


Figure 10: Effect of bin width on mean E2E latency.

in sparse state representations, which hinders the convergence of the RL algorithm. Conversely, an overly broad bin width obscures fine-grained load variations across different instances. Empirical results indicate that a bin width of 64 strikes the optimal trade-off between representation fidelity and learning efficiency, serving as our default configuration.

5.5 Ablation Studies

To quantify the individual contributions of the queuing and routing components, we run an ablation on the balanced mixed workload (ratio 0.33:0.34:0.33) across varying arrival rates, comparing three configurations: (1) **RAGS**: Full system (DynA + RL routing); (2) **RAGS w/o DynA**: FCFS queuing + RL routing; (3) **RAGS w/o RL**: DynA queuing + Round-Robin routing.

As shown in Figure 11, performance differences remain marginal at low arrival rates (RAGS: 11.206 s, RAGS w/o DynA: 11.205 s, RAGS w/o RL: 12.867 s) but widen significantly under heavy load. At the peak arrival rate, RAGS achieves 104.557 s, whereas removing DynA or RL routing degrades latency to 110.804 s and 148.758 s, respectively.

These results lead to three key insights. First, RL routing acts as the dominant contributor; omitting it increases peak mean latency by 42.27%, demonstrating that its benefits scale with system load. Second, DynA provides substantial gains at medium arrival rates, where its removal increases latency by 14.8% (20.782,s vs. 18.104,s),

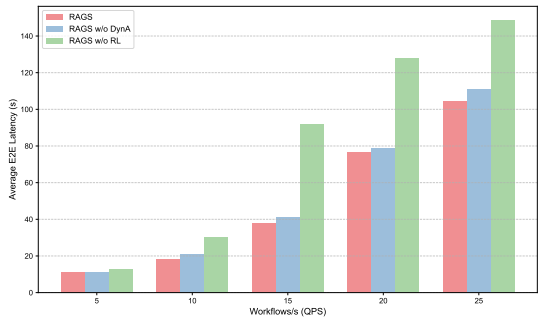


Figure 11: Component ablation under mixed workload.

confirming that progress-aware prioritization effectively mitigates head-of-line blocking. Third, the components are complementary: DynA accelerates critical requests into the routing stage, while the global RL policy prevents them from suffering secondary queuing within backend LLM engines.

5.6 Heterogeneous Cluster Evaluation

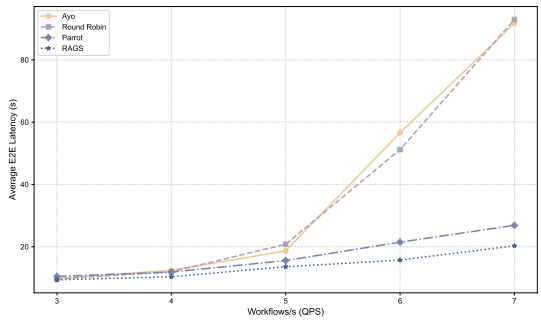


Figure 12: Heterogeneous GPU cluster evaluation results.

To validate RAGS on a heterogeneous GPU cluster, we conduct experiments using a 4-GPU setup comprising two NVIDIA A100s and two NVIDIA H100s, with meta-llama/Llama-2-7b-hf deployed across all instances. Because the A100 and H100 exhibit significant disparities in compute throughput and memory bandwidth, the system requires hardware-aware routing to prevent unbalanced allocations and resource bottlenecks.

Figure 12 illustrates the mean E2E latency across arrival rates (3–7 workflows/s). Round-Robin ignores hardware heterogeneity and distributes requests evenly, rendering the weaker A100s as system bottlenecks and degrading latency to over 80,s under high concurrency. Similarly, Parrot's hardware-agnostic static binding frequently overloads the A100 instances with compute-intensive requests.

In contrast, RAGS consistently achieves the lowest E2E latency. By jointly encoding GPU types and real-time instance loads within the RL state space, the routing policy implicitly senses hardware capacities and adaptively balances global traffic. At the peak rate of 7 workflows/s, RAGS reduces mean E2E latency by 30%–40%

compared to both Round-Robin and Parrot, confirming its robust generalizability in heterogeneous inference environments.

6 Related Work

LLM Inference Engines. vLLM [10] introduces PagedAttention to manage KV cache with OS-style virtual memory paging, significantly reducing memory fragmentation. SGLang [23] proposes RadixAttention, indexing historical KV caches in a radix tree to enable fine-grained cross-request prefix reuse and cache-aware routing. Sarathi-Serve [2] mitigates prefill and decode interference via chunked prefills, while Orca [22] pioneered continuous batching to maximize GPU utilization. Although effective for single-turn requests, these systems lack mechanisms to model the multi-step workflow dependencies inherent in Agent workloads.

LLM Agent Serving Systems. Parrot [11] pioneers agent workflow scheduling by introducing semantic variables that expose inter-request data dependencies to the serving layer, while employing static session affinity to improve KV cache reuse. Ayo [17] decomposes complex applications into primitive-level dataflow graphs , co-scheduling LLM and non-LLM stages to reduce end-to-end latency. Autellix [12] treats the entire Agent program as the scheduling unit, employing multi-level feedback queues based on accumulated critical-path execution time to mitigate head-of-line blocking. For caching, KVFlow [14] leverages reuse-distance estimations for predictive eviction and prefetching. Additionally, HexGen-Flow [16] introduces an SLO-driven scheduler with urgency-based preemption; yet, its coupling to Text-to-SQL pipelines severely limits its generalizability to arbitrary Agent topologies.

RL for Cluster Scheduling. DeepRM [13] and DL2 [15] demonstrate the effectiveness of deep RL for resource scheduling in DNN training clusters. TORTA [8] extends RL to LLM inference by dynamically adjusting geographic traffic routing ratios. Unlike these approaches, RAGS applies RL specifically to Agent workflow routing. By explicitly encoding workflow-level dependency states and KV cache residency, RAGS achieves an adaptive global routing policy that effectively balances hardware load with cache locality.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI)*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, Santa Clara, CA, 117–134.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021), 1–34.

[4] Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain. Accessed: 2026-03-13.

[5] Jinyuan Chen, Jiuchen Shi, Quan Chen, and Minyi Guo. 2025. Kairos: Low-latency multi-agent serving with shared llms and excessive loads in the public cloud. *arXiv preprint arXiv:2508.06948* (2025), 1–15.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021), 1–35.

[7] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. 2023. *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality*. https://lmsys.org/blog/2023-03-30-vicuna/

[8] Chengze Du, Zhiwei Yu, Heng Xu, Haojie Wang, Jialong Li, et al. 2025. Temporal-aware gpu resource allocation for distributed llm inference via reinforcement learning. *arXiv preprint arXiv:2507.10259* (2025), 1–16.

[9] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, December 10–16, 2023, New Orleans, LA, USA (NeurIPS)*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 18015–18027.

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles (SOSP)*, Jason Flinn and Margo I. Seltzer (Eds.). Association for Computing Machinery, Koblenz, Germany, 611–626.

[11] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, Santa Clara, CA, 929–945.

[12] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025), 1–17.

[13] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM workshop on hot topics in networks (HotNets)*, Bryan Ford and Alex C. Snoeren (Eds.). Association for Computing Machinery, Atlanta, GA, 50–56.

[14] Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *arXiv preprint arXiv:2507.07400* (2025), 1–12.

[15] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, Chen Meng, and Wei Lin. 2021. DL2: A deep learning-driven scheduler for deep learning clusters. *IEEE Transactions on Parallel and Distributed Systems* 32, 8 (2021), 1947–1960.

[16] You Peng, Youhe Jiang, Wenqi Jiang, Chen Wang, and Binhang Yuan. 2025. HEXGEN-FLOW: Optimizing LLM Inference Request Scheduling for Agentic Text-to-SQL. *arXiv preprint arXiv:2505.05286* (2025), 1–21.

[17] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS)*, Adrian Sampson, Martha Kim, and Christopher J. Rossbach (Eds.). Association for Computing Machinery, Rotterdam, Netherlands, 1302–1316.

[18] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*, Dale Schuurmans and Michael P. Wellman (Eds.), Vol. 30. AAAI Press, Phoenix, AZ, 2094–2100.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA (NeurIPS)*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V.Ñ. Vishwanathan, and Roman Garnett (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 5998–6008.

[20] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR), 2023*. OpenReview.net, Kigali, Rwanda.

[22] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI)*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, Carlsbad, CA, 521–538.

[23] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, December 10–15, 2024, Vancouver, BC, Canada (NeurIPS)*, Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 62557–62583.